

Web UI Automation Analysis

Anonymous^{1*}

^{1*}Department, AFFILIATION, Street, City, 100190, State, Country.

Corresponding author(s). E-mail(s): email@email.com;

Abstract

Based on a comprehensive multi-angle search across web UI automation, structured action prediction, retrieval-augmented agents, constrained decoding, and hard-negative training, here is the full analysis mapped to your pipeline.

Keywords: Web UI Automation, Agentic Workflow, Retrieval-Augmented Generation, Constrained Decoding

1 Top 12 Papers Table

#	Title / Venue / Year	Match	Key Idea	Pipeline Fit	Gain	Cost	Risk
1	WEPO: Web Element Preference Optimization (AAAI 2025)	5	DPO on hard negatives	src/train.py DPO	target_id	S	Low
2	Mind2Web (NeurIPS 2023)	5	Two-stage ranker	src/retrieval.py filter	target_id	S	Low
3	Agent Workflow Memory (arXiv 2024)	5	Workflow routine retrieval	src/retrieval.py index	exact	M	Low
4	RaDA (ACL 2024)	5	Two-stage retrieval	src/retrieval.py split	op/target	M	Low
5	REGENT (arXiv 2024)	4	Semi-parametric policy	src/retrieval.py dense	target_id	M	Med
6	Thinking Before Constraining (arXiv 2026)	4	Draft-conditioned decoding	src/inference.py FSM	format	S	Low
7	CoCo-Agent (ACL Findings 2024)	5	Op-conditioned prediction	src/train.py prefix	op/target	S	Low
8	ScribeAgent (arXiv 2024)	4	Domain-specific fine-tune	src/train.py data	exact	M	Low
9	WebCoT (arXiv 2025)	4	CoT distillation	src/train.py CoT	target_id	M	Low
10	diff History (arXiv 2023)	4	Unix-diff history	src/preprocess.py diff	target_id	S	Low
11	GTA1 (arXiv 2025)	4	Best-of-K sampling	src/inference.py voting	target_id	S	Low
12	FANTASE (arXiv 2024)	4	Trie-constrained decode	src/inference.py trie	format	M	Med

2 Prioritized Shortlist

3 Implement This Week

3.1 CoCo-Agent Conditional Decomposition (Paper 7)

- **Minimal experiment design:** Add a special `<op>` token as a CoT prefix before the JSON block. Train on your existing SFT data with a new output template: `<op>TYPE</op>{"op":"TYPE","target_id":"...","value":"..."}`. Compare exact match on a held-out validation split.
- **Required code changes:** `src/train.py`: update `output_template`, add `op-token` prefix loss weight $\times 2$. `src/inference.py`: parse `<op>` tag first, use predicted `op` to constrain the candidate search space (e.g., for CLICK, skip all `<input type=text>` elements from candidate list).
- **Failure risk:** If `op` prediction accuracy is already near-perfect in your data, decomposition adds no signal. Validate by checking how often `op` errors co-occur with target errors — if correlation is low, skip.

3.2 diff History Encoding (Paper 10)

- **Minimal experiment design:** Implement Python `difflib.unified_diff` over consecutive `cleaned_html` strings in history. Tokenize and measure average input length reduction vs. baseline. Test on validation set with same Qwen3-8B checkpoint, no retraining needed.
- **Required code changes:** `src/preprocess.py`: add `build_diff_history(steps: List[str]) -> str` function. Replace raw history concat with diff output. Add `max_context_length` guard to truncate oldest diffs first.
- **Failure risk:** If your HTML is already heavily cleaned/pruned, diff may be near-empty and uninformative. Fall back to raw history for first 2 steps (no previous state to diff against).

3.3 Best-of-K Inference with Consistency Guard (Paper 11)

- **Minimal experiment design:** For each test sample, generate $K=5$ JSON outputs (`temperature=0.7`). Apply your existing Consistency Guard to each; count valid outputs. Select the most frequent valid (`op`, `target_id`) combination. No model retraining required.
- **Required code changes:** `src/inference.py`: wrap single-call generation with `sample_k(prompt, k=5)`. Apply `consistency_guard(output)` to each. Implement majority vote over (`op`, `target_id`) pairs. Log how often the voted output differs from top-1 greedy.
- **Failure risk:** Inference cost $\times 5$. Viable only if GPU budget allows. Mitigate by triggering $K>1$ sampling only when confidence of greedy output is below threshold (e.g., top-token `logprob < -0.5`).

3.4 Draft-Conditioned Constrained Decoding (Paper 6)

- **Minimal experiment design:** Integrate `lm-format-enforcer` or `outlines` library with Qwen3. Measure: (a) format error rate before vs. after, (b) exact match change. Validate that reasoning quality is preserved by checking intermediate `<think>` content length.
- **Required code changes:** `src/inference.py`: detect boundary between `<think>...</think>` and the JSON block. Apply unconstrained decoding in `<think>` section; switch to JSON schema-constrained decoding after `</think>` token. Schema = `{"op": enum, "target_id": string, "value": string}`.
- **Failure risk:** Some constrained decoding libraries have poor performance with Qwen3 tokenizer. Test tokenization boundary alignment before full deployment.

4 High Upside, Medium Risk

4.1 WEPO-style DPO Hard Negatives (Paper 1)

- **Minimal experiment design:** For each SFT training sample, the gold candidate is the "chosen" response. Construct "rejected" responses by: (a) substituting the gold `target_id` with the 2nd or 3rd closest DOM-neighbor candidate, (b) substituting with a same-tag element from a different page section. Train DPO on top of SFT checkpoint for 1–2 epochs.
- **Required code changes:** `src/train.py`: add `build_dpo_pairs(sample)` that generates rejected completions. Implement DPO loss alongside CE loss with a mixing ratio ($\lambda_{DPO} = 0.3$ initially).
- **Failure risk:** If your candidate set is already highly curated, distance-based negatives may be too easy and provide no gradient signal.

4.2 Agent Workflow Memory Retrieval (Paper 3)

- **Minimal experiment design:** Cluster your training trajectories by site + task-template into 50–100 workflows. At inference, retrieve the top-1 matching workflow by (site, history signature) similarity. Inject as a prefix block.
- **Required code changes:** `src/retrieval.py`: add `WorkflowIndex` class. Index built offline. Injection prepended to similar-examples block.

4.3 WebCoT Rationale Annotation (Paper 9)

- **Minimal experiment design:** Use a larger teacher model (Qwen3-72B) to annotate 5–10K training samples with rationales. Fine-tune Qwen3-8B on these.
- **Required code changes:** `src/train.py`: update data pipeline to include rationale prefix in output template.

5 Direct Mapping to Your Code Modules

6 src/retrieval.py

- **RaDA Integration:** Split into `retrieve_task_exemplar` (task-level) and `retrieve_step_exemplar` (step-level).
- **AWM Integration:** Add `WorkflowMemory` class; gate injection by confidence score > 0.6 .
- **REGENT Integration:** Add dense retriever branch (bi-encoder 384-dim embedding + FAISS index).

7 src/preprocess.py

- **diff History:** Add `build_diff_history` using `difflib.unified_diff`.
- **Candidate Ranker:** Add `rank_candidates(task, history, candidates)` using DeBERTa cross-encoder; filter to top-K=10.

8 src/train.py

- **WEPO DPO:** Add `build_dpo_pairs`; use `DPOTrainer` with $\beta = 0.1$.
- **CoCo-Agent:** Modify `output_template`; weight op-token loss $\times 2$.
- **ScribeAgent:** 70% workflow data / 30% real_web data; 8K context; LR $1e-5$.

9 src/inference.py

- **DCCD Decoding:** Add `draft_conditioned_decode` using outlines or `lm-format-enforcer`.
- **GTA1 Best-of-K:** Add `best_of_k_inference`; log activation rate.
- **FANTASE Trie:** Add `build_valid_action_trie` for per-sample FSM construction.

10 Ablation Plan

#	Ablation	Metric	Direction
A1	Remove order aug	target_id accuracy	Drop 3-8%
A2	Add DPO negatives	target_id accuracy	Gain 5-12%
A3	Full raw history	target_id accuracy	Drop 3-6%
A4	Add pre-ranker	target_id accuracy	Gain 4-9%
A5	CoCo 2-stage	op/target accuracy	Gain
A6	Constrained decode	exact match	Format error $\rightarrow 0\%$
A7	Best-of-K (K=5)	exact match	Gain 2-5%
A8	Workflow retrieval	exact match	Gain 8-15%

11 Rejection Filter — 5 Low-ROI Directions

1. **Visual Grounding:** Vision encoders add significant overhead; unless switching to vision-native agents, the ROI is low for HTML-rich pipelines.
2. **Online RL:** Requires complex sandbox and 100x compute; SFT/DPO has much more headroom at your current stage.
3. **World Models:** Transition-focused abstractions are complex and target planning horizon, not element selection.
4. **Tool-Use Abstraction:** Architectural pivot that invalidates training data and benchmarks.
5. **Generic Context Compression:** Diff-history is already a near-free, efficient solution for your specific needs.